Enforcing The Chinese Wall Security Policy in Security-Enhanced Linux

by

Calum Cardownie

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project

University of Strathclyde Glasgow

Department of
Computer and Information Sciences

March 2026

Abstract

The abstract is a bird's eye view of the project. It should not exceed one page. Mention the scope and objectives of the project, the methodology used,the main findings, and significance of your results.

Acknowledgements

I would like to thank

Contents

1 Introduction **1**
1.1 Aim . . . 1
1.2 Objectives . . . 2
1.3 Report Format . . . 2

2 Background Research **3**
2.1 The Chinese Wall Policy . . . 3
2.2 The Chinese Wall in Adjacent Systems . . . 4
2.3 Security-Enhanced Linux . . . 6

3 Specification **10**
3.1 Methodology . . . 10
3.2 Analysis . . . 10
3.3 Requirements . . . 11

4 Design **13**
4.1 Technology Selection . . . 13
4.2 Environment Design . . . 13
4.3 System Design . . . 13

5 Product **15**
5.1 Implementation . . . 15

6 Verification & Validation **16**
6.1 Testing Suite . . . 16
6.2 Testing Constraints . . . 16

7 Results & Evaluation **17**
7.1 Evaluation Process . . . 17
7.2 Results of Evaluation . . . 17
7.3 Returning to the Research Questions . . . 17

8 Discussion & Reflection **18**
8.1 Interpreting the Results . . . 18
8.2 Reflection . . . 18
8.3 Challenges . . . 18
8.4 Limitations . . . 18
8.5 Future Work . . . 18

9 Conclusion **19**

A Appendix **21**
A.1 Marking Scheme . . . 22

1 Introduction

1.1 Aim

Implementation of the Chinese Wall (Brewer-Nash) Model, by leveraging the mandatory access control system introduced by Security-Enhanced Linux, is the primary goal of this project. SELinux is a well-known solution to security concerns threatening servers and personal devices. SELinux implementing Red Hat (IBM) have solidified themselves as the industry choice for secure hosting solutions, being a leading operating system used by over 90% of Fortune 500 companies for its quality privacy guarantee [2].

The Chinese Wall's dynamic nature makes it an intriguing security policy, which let it stand in stark contrast from typical static security policies. For this reason or others, it has received a noticeable lack of attention in the space surrounding SELinux. While providing an implementation for SELinux, this report delves into the use cases of this specific policy, where it has succeeded in the past, and what its inclusion in SELinux allows.

"The more you know, the less you can share" serves the basis for the policy, where a subject (a process, daemon or user) in the system loses more privileges as it completes its lifecycle. Conflict of Interest (`CoI`) classes dictate the main restrictions, preventing the subject from sharing sensitive information between objects (files) that are conflicting with each-other. A lattice based system is used to transition subject labels based on their read and write history accordingly, which should suffice to handle all cases of the Chinese Wall.

Previous research into both Chinese Wall effectiveness and SELinux was factored in the design process, along with hands on experimentation and prototyping of different solutions. The development of the Chinese Wall daemon should serve to open possibilities of dynamic policies in the otherwise static SELinux, and potentially lead the way in future security solutions.

To enforce this policy model in SELinux, policy modules must be compiled into the kernel via the security server and work in tandem with the designed Chinese Wall Security Daemon, developed to be capable of enforce dynamic label alterations to monitored subjects in a directory. Finally, this allows the SELinux Access Vector Cache (AVC) to handle the restrictions on a kernel level, successfully locking the system down to `CoI` breaches.

1.2 Objectives

1.1 Create a suitable virtual environment for SELinux.

1.2 Understand the principles of:

1.2.A The Chinese Wall security model.

1.2.B SELinux's security server & type enforcement system.

1.3 Implement a static implementation of the policy, confined by SELinux.

1.4 Develop a system daemon capable of monitoring subjects and their operations on objects.

1.5 Update said daemon to dynamically alter labels, allowing SELinux to enforce the Chinese Wall policy.

1.6 Fully integrate the Chinese Wall Daemon into SELinux's environment, including its own policy module, file hierarchy compliance and good UNIX citizenship.

1.7 Analyse the difficulties that arose with development and conclude why development in this area has been lacklustre.

1.3 Report Format

The report is structured in an sequence appropriate to the process undertook addressing this challenge, detailing background research, solution design, implementation and testing, followed by results and evaluation.

2 Background Research

2.1 The Chinese Wall Policy

The Brewer-Nash Model

The Chinese Wall Security Model - or Brewer and Nash Model - is a security model that prevents information from conflicts of interest (`CoI`) groups being spread [5]. As opposed to the classic multilevel security Bell LaPadula model, which enforces *"no read up, no write down"* [4], the Chinese Wall model allows for information flow freely until a subject attempts to read from a `CoI` object. Upon this access, the system dynamically revokes their write permissions to other objects in the `CoI` group, creating a "wall" in-between them. Consider the following scenario:

A data server stores information on Bank A and B, and Tech Company A and B. These are in a `CoI` relation with their respective counterpart.

If a user approaches the dataset, they are initially able to access any object. As far as the system is concerned, this user is free as they have no prior knowledge of either companies. Suppose the user has read from Bank A, the situation develops - the user now possesses information that, deemed by the Chinese Wall, should not be allowed to reach Bank B, as they in a `CoI` relation with Bank A. This is where the wall revokes the write access from the user in regard to Bank B. Now information *cannot* flow from Bank A to B. The system is constructed to definitively prevent this leakage scenario. It should also be noted that once information from Bank A is read, it cannot be shared to Tech Company A or B. In that scenario, a different subject could read from Tech Company A and write to Bank B, as they are not in conflict. The model

The Brewer-Nash Model Intricacies

The Brewer-Nash Model provides a mathematical format that is based on the larger Chinese Wall policy. The rules - while open ended - demonstrated the Chinese Wall as a more mathematical concept rather than a policy. They broke it down into the following rules, inspired by Bell-LaPadula:

- **BN Simple Read Rule**: Subject `S` can read object `O` only if `O` is in the same Company Dataset (`CD`) as an object already read by `S`, or if `O's CD` is in a different Conflict of Interest group from all `CD`s of objects already read by `S`.

- **BN Write Rule (* Property)**: Subject `S` can write to an object `O` only if `S` can read `O` **and** all objects `O'` read by `S` are in the same `CD` as `O`.

Brewer-Nash's model was left more open-ended than would be expected from an influential report. For one, readers are split on interpretations of certain definitions in the paper (see Def.1 and Axiom 6 in the original appendix of Brewer & Nash). As such, this report will outline an understanding of it and follow that.

After the Brewer-Nash model was communicated, another paper was published once again by Brewer & Nash, but also T.Y. Lin, who presented them with potential improvements on their model after their original paper, leading to the Aggressive Chinese Wall [11]. This idea, trying to disprove the assumption that the conflict of interest is an equivalent relation, has been accepted into the mainstream since it was proposed.

2.2 The Chinese Wall in Adjacent Systems

Chinese Wall in Unix

A Chinese Wall model implementation was made in standard Unix, leveraging its Discretionary Access Control (DAC) security system also adopted by Linux and macOS. The 1997 Unix proof by S.N. Foley built upon T.Y. Lin's Aggressive Chinese Wall Model, illustrating the possibility of its creation in with DAC [7]. The Unix proof features a `CoI` matrix, detailing a non-transitive version of the `CoI` relations. Foley's concept focused on the restriction of users, preventing a user of the computer from spreading information enforced via their user ID and group ID. DAC is defined (simply) such that objects and their privileges are managed by their creator, and the super-user (root, or admin) can access and alter anything. DAC enforces based on the user ID and any groups with which they belong. While DAC brings about great flexibility, the models' reliance on ownership often leads to uncontrolled privilege propagation. It is known to be difficult to measure the security of the DAC model, and in some cases it is undecidable [6]. This indicates there is research, but there is a lack of documented attempts to specifically implement it into the Unix or adjacent kernel.

Hypervisor-Level Chinese Wall

Chinese Walls have been implemented in Cloud Computing against many threat models. Cloud Computing involves many Virtual Machines (VMs) running on one core system, with the virtual

machine monitor, or *"hypervisor"* acting as the delegating force of the computers' hardware to the VMs. Each VM is a separate Operating System (OS), being completely independent of the other VMs on a machine.

The sHype Mandatory Access Control system was added to the Xen hypervisor to defend against information sharing between VMs by R. Sailer et al [12]. The hypervisor prevents inter-VM communication with kernel hooks, intercepting and evaluating every attempt to communicate across VM's. Whilst claiming to be a Chinese Wall, the sHype architecture does not truly capture the notion of Brewer & Nash's Model, instead focusing on static Chinese Wall types (CW-types). Regardless, sHype does successfully prevent against `CoI` leakages, in the case of insider activities or inter-VM attacks.

Inter-VM attacks can happen if a VM on the system is breached, and can allow a targeted attack on another VM, or to even compromise the hypervisor. To eliminate the possibility of inter-VM attacks from competitors, a different Chinese Wall security model was implemented. The Chinese Wall Central Management System (CWCMS), operating above the Kernel-base Virtual Machine level via installed control agents, with a sole focus of actively preventing `CoI` breaches was designed by Tsai et al [13]. Once a VM in a `CoI` group is running on a physical host, the Wall prevents other VMs belonging to the same `CoI` group from being scheduled on the same machine in advance. This is a pre-arranged decision made by graph colouring, as opposed to sHype which simply attempts to deploy it on different systems until one is satisfactory. This effectively eliminates the risk of inter-VM attacks in these systems. Similar to sHype, CWCMS also lacks dynamic label profiling, resolving to a statically enforced, yet successful, policy model.

Finally, a dynamic Wall was created in Yu et al's SVMS system [14]. SVMS eliminates intrahost *side channel attacks* (SCA), where malicious users extract private information by reading from shared data caches on the host machine. They use a combination of the tried and trued 'prevent simultaneous execution', but also a more nuanced advance with VM placement and migration solutions. Using VM traces accumulated by tracking the *aggressive conflict of interest relation* (ACIR) and *aggressive in ally with relation* (AIAR), they create a dynamic policy model faithful to the Brewer-Nash model, proving that the rules meet the simple-security property and *-security property.

All models, while different, agree that the *"Chinese Wall"* involves preventing conflicting resources from running at the same time. Whilst not exactly cohering to the Brewer-Nash model, they all

manage to succeed in their endeavours to create a Chinese Wall.

Chinese Wall in Workflow Management

Beyond virtualized environments, the Chinese Wall model has also successfully been implemented in workflow management systems. Workflow Management Systems (WfMS) are intrinsically network-based applications, potentially hosting multiple conflicting sources of information. As such, a secure WfMS must implement security features, specifically targeting information confidentiality as a main point of effort. The Chinese Wall security model can be leveraged to prevent conflict of interest leakage in this environment. The authors proposed a Chinese Wall Security Policy Specification Language (CWSPSL) that they then implemented with Java [9]. The paper mentions that RBAC is unable to implement general cases of the Chinese Wall, as it has issues implementing the BN read rule in dynamic contexts. The dynamic nature of the Chinese Wall is proven to be difficult to implement with standard tradition.

Chinese Walls in Financial Institutions

Chinese Walls serve as a pillar of compliance in the financial sector. The ethical policy, more commonly just referred as *The Wall* in finance, is a commonplace measure that should theoretically and practically prevent misuse of information leakage in banks [8] and other financial management firms. Thus, the Wall is more attuned to person-level restrictions as opposed to system-wide enforcement as is present in modern considerations of the Wall. While entirely different from modern systems, it still acts on 'The more you know, the less you can share'

2.3 Security-Enhanced Linux

Mandatory Access Control System

Security Enhanced Linux implements a Mandatory Access Control (MAC) system, providing a policy-driven security layer extended beyond the classic DAC. Utilisation of labels - applied to both Objects and Subjects, and verbose policy modules provide the OS great freedom in restriction and enforcement. Labels are devised in the form `user_u:role_r:type_t:s0`. For instance, a simple policy may be that an object of type `httpd_t` can only be operated on by a subject of type `httpd_t` and not `sshd_t`. This provides strong mitigation against broken access control damages, as subjects

can only access objects that they are contained to by SELinux's security server. In a case where a subject gets hijacked, it is still only restricted to the domain it originally had on the system.

Parts at Play

Being based off of the Flux Advanced Security Kernel (FLASK) security architecture for flexible nondiscretionary access controls, the architecture of SELinux is split into 2 main bodies:

- Policy enforcement via the Access Vector Cache (AVC)
- Policy decision-making via the policy repository and security server

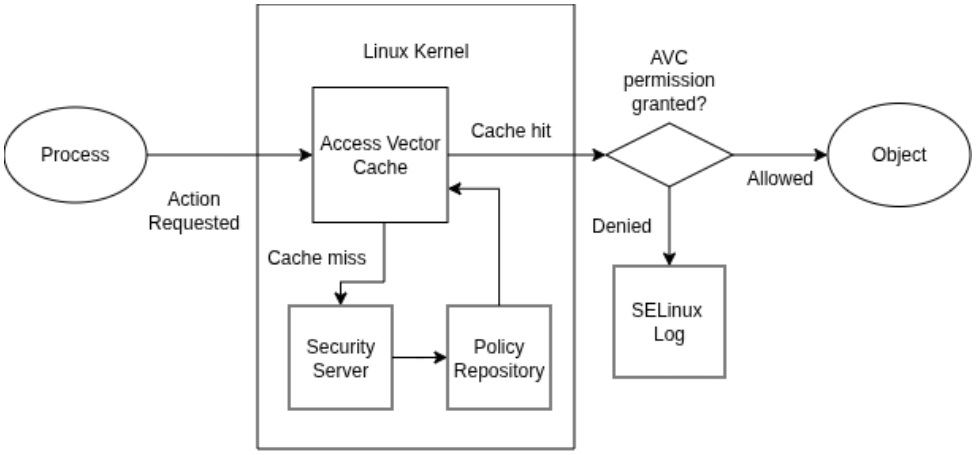

Figure 1: SELinux Components

Policy Module System

The policy modules that SELinux provides can be controlled solely by the super-user. Red Hat 9 by default has **440** modules, ranging every domain in the system. The *policycoreutils* package introduces developer tools to install and alter policy modules on the system. With it, a policy module can be generated by calling *sepolicy generate –init* from the command line. The boilerplate includes:

- Type Enforcement *(.te)* file: Unique syntax notates dependencies and designs rules, introduced to restrict the new labels.
- File Contents *(.fc)* file: Details the file contents related to the policy.
- RPM Spec *(.spec)* file: Installs the policy into the kernel, including labelling, the interface file and associated man page of the policy.

- Interface *(.if)* file: Defines the interfaces for the types generated in the type enforcement file, allowing policies to reference each-other.
- Shell Script *(.sh)* file: Finally, this script can be run with *sudo* to compile and install the policy module. Under the hood, various commands are executed, including: compiling a *.mod* file, *.pp* file, then installing the *.pp* file with *semodule -i* finally using *restorecon* to repair the file context of the designed module.

Once installed, policy modules will survive reboots in the system and restrict based on their domain while SELinux is in *Enforcing* mode.

Origin of Linux & SELinux

SELinux is adopted by many distributions of Linux, including Red Hat Enterprise Linux (RHEL), Fedora and even mobile operating system giant Android. The origin of SELinux can be attributed to the National Security Agency's Trusted Unix (`TRUSTIX`) Working Group's workings on a series of books called the Rainbow books - A series of Computer Security standards and guidelines by the United States Government. They published one and created a second (unpublished) book that included a formal SELinux model and associated evaluation evidence prototype. SELinux was a demonstration that mandatory access control was viable and could be added to the Linux kernel.

Linux, as an OS was able to adopt this because of its commitment to software freedom and ethical use of technology, originally being an open-source derivation from the Unix that was locked down by AT&T's IP. Linus Torvalds (also credited for the creation of git) combined his Linux kernel with the GNU system tools by the GNU Project (Richard Stallman and various developers) to create GNU/Linux. This open-source philosophy was a key factor enabling SELinux's integration into the mainline kernel in the 2.6 series in December 2004. The impact of Linux is huge, with 85% of smartphones [10] and 96% of top 1 million websites [1] powered by the free OS.

Red Hat Enterprise Linux

Tech Giant IBM produced RHEL has a 33% market share in worldwide operating systems [3], no doubt because of their commitment to reliability, stability and security.

As one of many distributions of Linux, RHEL is a unique as a subscription based distribution, with

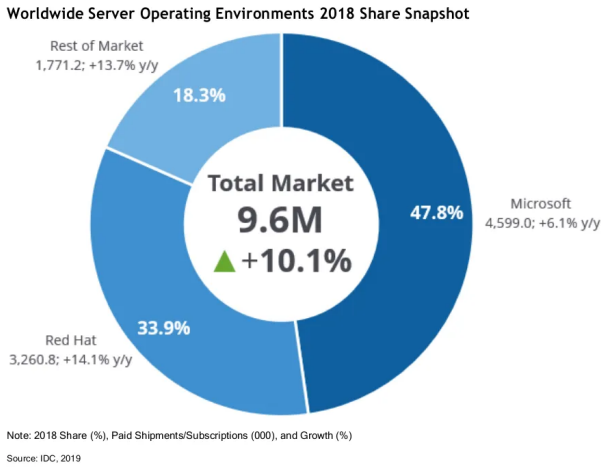

Figure 2: Red Hat statistics

a target audience of companies expecting nothing less than the finest. This promotes SELinux as a lucrative avenue for development.

This section should provide the grounding for your project. You will be able to refer back to this in Section 3, where you report on design decisions and methodology. There should be a strong link to your research questions (for project types 2 and 3). There will be a strong link to the software design decisions for project type 1. Don't just include a paragraph for each paper you read. Synthesize it and create a story line. Review between 5 and 10 authoritative sources. For research based projects, some of these should be from peer-reviewed journals or conferences. This section will reference literature like this [?]. Sloppy referencing will (1) take time and effort, and (2) lose you marks if you don't do it.

3 Specification

3.1 Methodology

The decision to adhere to a methodology early in development helps set up consistent and satisfying progress over the life of the project. As this project was undertaken as a solo developer, the decision of methodology was not as crucial as a team setting. Still, the project lifecycle followed is more closely attuned to the agile principle.

3.1.1 Agile

Agile methodology focuses on multiple iterative stages, constantly evaluating the progress and direction of the project, and tuning decisions to reach the solution. As SELinux is a system with predefined capabilities and limitations, various different methods were used throughout to understand the boundaries and prototyping was updated accordingly. Experimentation in SELinux was carried out simultaneously alongside research, allowing accumulated insight to be backed by personal experience, and a more complete and satisfying solution.

This agile process was accompanied by a simple kanban board, easily conveying the stage of the project as well as next steps and currently occupied tasks. This allowed for a more streamlined lifecycle of an idea.

3.1.2 Waterfall

Waterfall involves a predefined set of requirements, of which progress is made solely to the final objective. This methodology benefits experience, where familiarity with the project environment allows decisions to be made far in advance. Waterfall usually has issues with bugs, as testing is only a focus in the later stages of development. As this is a project within time constraints and a single developer, agile was selected.

3.2 Analysis

To enforce a dynamic security policy, its natural to deduce that some sort of process must be constantly running to be capable of enforcing said dynamism. During experimentation on the boundaries of SELinux, it was surmised that the most effective way to leverage its MAC system was to change the type fragment of the labels on both subjects and objects. The user, role and

category of the subjects/objects were not sufficient to enforce a security policy, as policy modules were unable to be compiled to enforce restrictions based entirely on said label fragments.

Continuous experimentation on the type fragment of the label preceded the philosophy that would be the main premise of the Chinese Wall Daemon. Pre-compiled policy modules for all scenarios, and a daemon that altered the labels of subjects at runtime according to their behaviour and the Chinese Wall policy. This daemon would also need to be fully integrated into SELinux's system, running under its own `cwalld_t` type rather than `unconfined_t`.

Policy modules are unable to be added to the SELinux security server at runtime, as it requires a complete re-compile of the security server and kernel. This operation is capable of taking anywhere between 4 and 8 seconds - suffice to say unrealistic at runtime for something intended as a background process.

3.3 Requirements

Requirements specified early in the agile cycle follow the MoSCoW principle. Requirements are split into 4 categories:

- Must have - features that must be implemented to achieve the goal.
- Should have - features that should be implemented to develop the solution.
- Could have - features that could be implemented but are not priorised.
- Won't have - features that were considered but decided against.

Functional requirements:

Non-functional requirements:

These requirements were solidified after considering the functionality that would be needed to enforce the Chinese Wall. While the requirements were decided from the beginning, the means of which to complete them was left ambiguous, in line with agile structure.

Code	Requirement	MoSCoW
FR1	`cwalld` must alter necessary configurations in the system before running.	**Must**
FR2	`cwalld` must monitor all operations made by subjects across a given directory.	**Must**
FR3	`cwalld` must make decisions based on subject label and object label aligned with the Chinese Wall policy.	**Must**
FR4	`cwalld` must alter the labels of subjects at upon call.	**Must**
FR5	`cwalld` must be accompanied by policy modules for objects.	**Must**
FR6	`cwalld` must be accompanied by policy modules for subjects.	**Must**
FR7	`cwalld` should be confined by SELinux with its own custom policy.	**Should**
FR8	`cwalld` should run as a service in the system with its own service declaration file.	**Should**
FR9	`cwalld` should write all operations to a log.	**Should**
FR10	`cwalld` should include a binary to follow and display the log in real time.	**Should**
FR11	`cwalld` could automatically create necessary policy modules for its runtime.	**Could**
FR12	`cwalld` could automatically install necessary policy modules before running.	**Could**
FR13	`cwalld` could dynamically enforce the policy given a set of `CoI` classes as input.	**Could**
FR14	`cwalld` won't enforce the Chinese Wall policy on the system-wide.	**Won't**

Code	Requirement	MoSCoW
NFR1	`cwalld` shall solely be controlled by the super-user.	**Must**
NFR2	`cwalld` shall run as a daemon in user-space of the computer.	**Should**
NFR3	`cwalld` shall be written with structural, intentional and idiomatic code.	**Should**
NFR4	`cwalld` shall be a good UNIX citizen, following classic system conventions.	**Should**
NFR5	`cwalld` shall function as intended across both x86 and ARM architecture.	**Should**
NFR6	`cwalld` shall act atomically, ensuring the Chinese Wall is instantaneous.	**Could**

4 Design

Describe all details of the design and procedures used to achieve the project objectives. Do this chronologically.

4.1 Technology Selection

Go *(or Golang)* was chosen for the development of the Chinese Wall Daemon, due to its strong affinity with system-level programming. Go's advantages in systems engineering range from such:

- **Systems Origin:** Go was developed with systems programming at its core, remaining unpunishing while giving the developer great versatility and library access.
- **Performance:** As a compiled language, the flexibility to be built into a binary means minimal overhead at runtime. Go promises low-latency, high-performance applications.
- **Modernity:** Go is a modern alternative to the otherwise stagnant systems engineering environment. The focus on clean, idiomatic code with statically enforced typing means debugging can be trivial if common practice is followed.
- **Familiarity:** The simplicity of Go discourages unnecessary complexity while promoting hands-on learning. Repeated experience working with the language supported the choice and guaranteed steady progress in the venture.

Narrowly beating out C for the chosen language, Go proved to be the correct choice. It's simplicity proved that navigating the complex SELinux environment would be possible with patience.

4.2 Environment Design

The SELinux AVC makes all its decisions based on the precompiled policy modules.

4.3 System Design

Show how you designed your database (if appropriate) and how you designed your system architecture, and the individual parts. Use UML and an Entity Relationship diagram

It should be detailed enough to allow for an assessment of the rigour of your process, and, in the case of research projects, in terms of how well grounded your research is in the

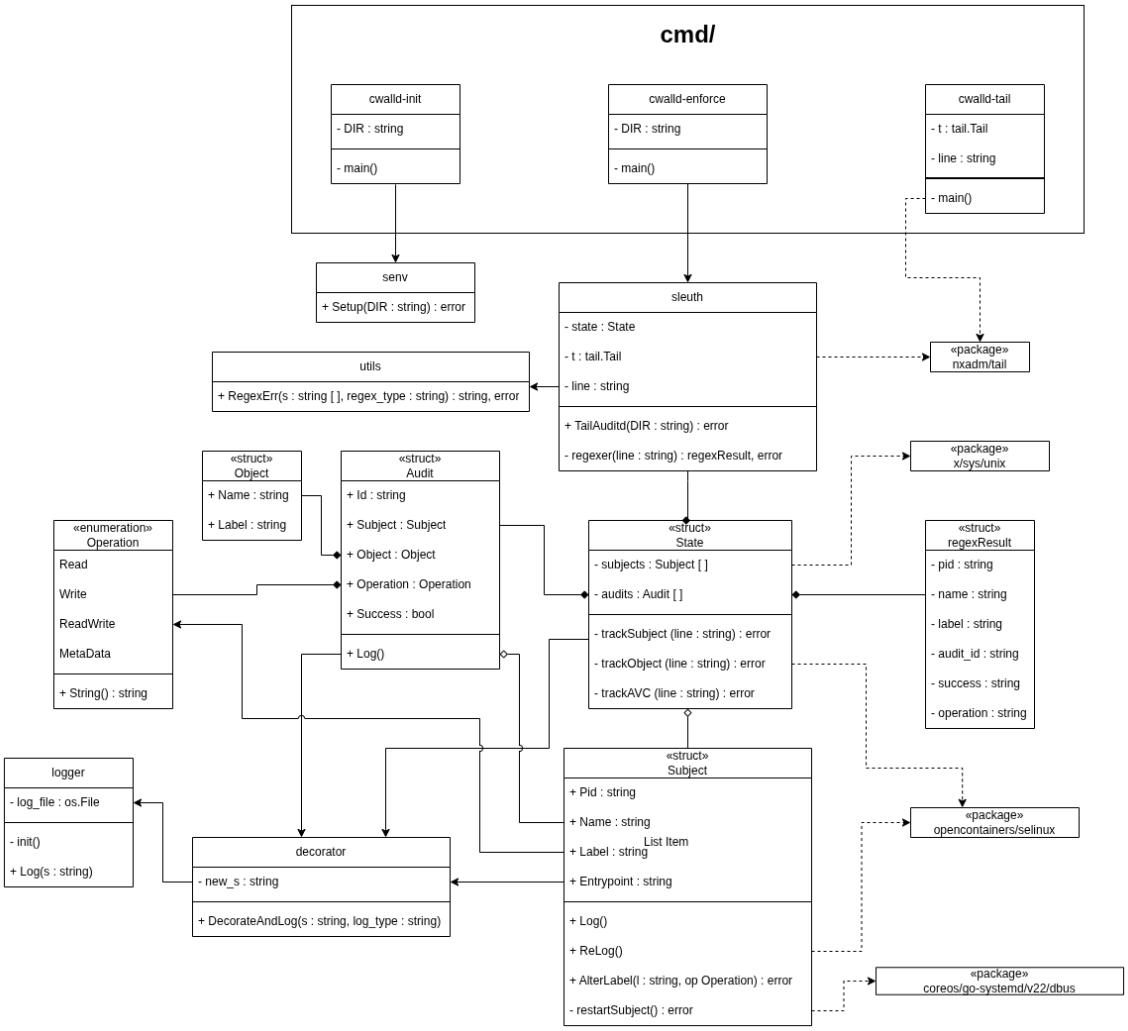

Figure 3: cwalld UML diagram

research literature. In these cases, refer back to relevant sections in the previous chapter. Say which software lifecycle approach you used e.g., Waterfall, Spiral, Agile. How did you gather user requirements?

5 Product

5.1 Implementation

Provide implementation details: language used, architecture (e.g. server and client, or hub and spoke). Explain how you secured personal details.

6 Verification & Validation

6.1 Testing Suite

6.2 Testing Constraints

How did you verify that your software was debugged and worked correctly? How did you validate that you had implemented all the functional and non-functional requirements?

7 Results & Evaluation

Results are often presented in tables, figures and other relevant illustrations. Include text that refers to these figures/tables.

7.1 Evaluation Process

If you involved humans in the evaluation, how many did you have? What can you say about the demographics of your participants? (If you did collect these) In terms of the user interface, how did you carry out a usability evaluation, how did you go about doing this? How did you recruit participants? In terms of delivering functionality, did you carry out user acceptance testing? (see attached guidance).

7.2 Results of Evaluation

This Section includes a direct interpretation of the gathered data and evaluation processes.

7.3 Returning to the Research Questions

Return to research questions or objectives as appropriate.

7.3.1 RQ1

It is clear from our findings that James Bond was born in Wigtown in Scotland. However, he grew up in Diss, in Norfolk. We know this because ....

7.3.2 RQ2

We were not able to answer this question from our studies, although some suggestions were made. These could not be proven.

7.3.3 Objective 1

7.3.4 Objective 2

8 Discussion & Reflection

8.1 Interpreting the Results

Here you will discuss your findings. This is especially relevant for research projects. You might interpret what the data and evaluation implies, both for future research and for practice (if appropriate). The discussion is not a review of literature. You should try to compare research findings with previous work, provide explanations for your findings, discuss research findings, in terms of their contribution.

8.2 Reflection

Look back and think about what you would do differently if you were going to start the project with the knowledge you have now. Be honest about your mistakes or missteps.

8.3 Challenges

This is not the place to mention personal circumstances but rather challenges related to the work involved in the project.

8.4 Limitations

Acknowledge things like: small number of participants, software wasn't completely debugged, or whatever else went wrong and affected your project. Include as appropriate

8.5 Future Work

If someone else wanted to build on your project's product, what would be cool to do next?

9 Conclusion

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS. Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

References

[1] Can the internet exist without linux? | ZDNET.

[2] Red hat enterprise linux | red hat partner connect.

[3] Red hat: Leading the enterprise linux server market.

[4] D. E. Bell and L. J. Lapadula. Secure computer systems: Mathematical foundations. Number: MTR2547VOL1.

[5] D. Brewer and M. Nash. The chinese wall security policy. In *Proceedings. 1989 IEEE Symposium on Security and Privacy*, pages 206–214. IEEE Comput. Soc. Press.

[6] Y. Fan, Z. Han, J. Liu, and Y. Zhao. A mandatory access control model with enhanced flexibility. In *2009 International Conference on Multimedia Information Networking and Security*, volume 1, pages 120–124. ISSN: 2162-8998.

[7] S. N. Foley. Building chinese walls in standard unixTM. 16(6):551–563.

[8] L. Herzel and D. E. Colling. The chinese wall and conflict of interest in banks. 34(1):73–115. Publisher: American Bar Association.

[9] Y.-C. Hsiao and G.-H. Hwang. Implementing the chinese wall security model in workflow management systems. In *International Symposium on Parallel and Distributed Processing with Applications*, pages 574–581. ISSN: 2158-9208.

[10] H. James. 85% of all smartphones are powered by linux.

[11] T. Y. Lin. Chinese wall security policy-an aggressive model. pages 282,283,284,285,286,287,288,289–282,283,284,285,286,287,288,289. IEEE Computer Society.

[12] R. Sailer, T. Jaeger, E. Valdez, R. Caceres, R. Perez, S. Berger, J. Griffin, and L. van Doorn. Building a MAC-based security architecture for the xen open-source hypervisor. In *21st Annual Computer Security Applications Conference (ACSAC'05)*, pages 10 pp.–285. ISSN: 1063-9527.

[13] T.-H. Tsai, Y.-C. Chen, H.-C. Huang, P.-M. Huang, and K.-S. Chou. A practical chinese wall security model in cloud computing. In *2011 13th Asia-Pacific Network Operations and Management Symposium*, pages 1–4.

[14] S. Yu, X. Gui, J. Lin, F. Tian, J. Zhao, and M. Dai. A security-awareness virtual machine management scheme based on chinese wall policy in cloud computing. 2014(1):805923. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1155/2014/805923.

A Appendix

This is where you can include your documentation. Remember that the marker is not required to read this, but might well check to ensure that you have included product documentation, and ethical approval, as required.

A.1 Marking Scheme

REMEMBER TO DELETE THIS. IT IS ONLY INCLUDED FOR your INFORMATION.

11.2. Marking Schemes

There are three marking schemes which weight the assessment criteria as follows.

	Software Development Based	**Experimentation-based with Significant Software Development**	**Experiment-based**
Project progress presentation	10%	10%	10%
Product			
Implementation (including documentation as this indicates maintainability)	25	20	10
Verification and Validation	10	5	5
Product Total:	35%	25%	15%
Process			
Methodology, analysis and documentation	15	20	20
Design	10	5	5
Process Total	25%	25%	25%
Results and Evaluation			
	15%	25%	35%
Report Presentation			
	10%	10%	10%
Student Performance			
	5%	5%	5%